Project 1: RAG and Context Engineering

1 Learning Outcomes

By completing this project, you will:

1. Build an end-to-end RAG pipeline from scratch, including chunking, embedding, indexing, retrieval, reranking, and LLM-based generation.
2. Create high-quality evaluation data (golden Q&A pairs) and understand the principles of data curation for AI applications (Topic 1).
3. Understand and apply evaluation metrics for both retrieval quality and generation quality, including LLM-as-judge evaluation, and optionally design your own custom metrics (Topic 1).
4. Use RapidFire AI OSS to run systematic multi-config experiments, compare RAG configs side by side, and converge on better outcomes through disciplined experimentation (Topic 2).
5. Develop engineering judgment about retrieval and context engineering tradeoffs that transfer to real-world AI applications (Topic 2).

2 High-Level Description

You will build a single-turn Q&A system over a technical documentation corpus (the RapidFire AI OSS documentation) and use RapidFire AI's multi-config experimentation framework to systematically explore and optimize your RAG pipeline.

Your system must operate within a per-query context budget of **2,000 tokens** passed to the LLM, including system prompt and any other content in the context. This constraint emulates realistic production conditions wherein document corpora are often far larger (billions of tokens) than any LLM's context window, making retrieval engineering essential, or large enough to be too expensive, too slow, or too error-prone to ingest entirely into the context for every query. Thus, this constraint here ensures you learn the same retrieval engineering skills on a manageable corpus, analogous to learning to implement schedulers or data structures at smaller scales.

The project has three main components:

1. **Golden Data Creation:** Create at least 20 golden-labeled Q&A pairs over the documentation corpus, with identified source locations.
2. **System Building:** Build a RAG pipeline that, given a user question, retrieves relevant document chunks and generates an answer within the context budget.
3. **Experimentation and Evaluation:** Use RapidFire AI OSS to systematically compare multiple RAG configurations (varying chunking, embedding, retrieval, reranking, and prompting strategies) and analyze results.

3 Infrastructure

- **Compute:** UCSD DSMLP Resources: Datahub CPU-only instances and LLM access via TritonAI LLM gateway. Specific access instructions for this edition provided via Piazza. Access control and resource availability managed by UCSD ITS.

- **Experimentation Framework:** RapidFire AI OSS: `pip install rapidfireai`. Use its `run_evals` mode to evaluate multiple RAG configs in one go.
- **RAG Pipeline:** RapidFire AI wraps LangChain's RAG abstractions via `RFLangChainRagSpec`, which covers document loading, chunking, embedding, indexing, retrieval, reranking, and context serialization. LangChain is the recommended (and natively supported) library for building your pipeline.
- **Embedding Models:** OpenAI embedding models or OSS models that run on CPUs only, e.g., `HuggingFaceEmbeddings` with sentence-transformers, BGE, etc.

Getting Started with Datahub, LLM Gateway and RapidFire AI

- **Datahub**: Access DSMLP Datahub at https://datahub.ucsd.edu/. Sign in with your UCSD SSO credentials and launch the 8 core, 32GiB RAM instance for the project.
- **TritonAI LLM Gateway**: Your TritonAI API key is available in `api-key.txt` inside your user home directory. Available models are listed at https://tritonai-api.ucsd.edu/ui/model_hub_table/. The gateway API is OpenAI-compatible; example code snippets are available by clicking a model name in the table.
- **RapidFire AI**: Before starting the project, review the RapidFire AI overview, its RAG walkthrough, and the accompanying tutorial Colab notebook.

Vector Store Options

RapidFire AI supports three vector store backends:

- **FAISS** (default): In-memory, no setup required. Operates only in Create mode (rebuilt each run). Recommended for getting started quickly.
- **PGVector** (PostgreSQL): Persistent vector store. Supports Create, Read, and Update modes. In Read/Update mode, provide a pre-existing `collection_name` and connection string.
- **Pinecone:** Cloud-hosted persistent vector store. Supports Create, Read, and Update modes. In Read/Update mode, provide a pre-existing `index_namespace`. Requires a Pinecone API key (free tier available).

Using PGVector or Pinecone in Read mode is especially useful for efficient experimentation: you preprocess and embed your corpus once, then run many retrieval/generation configurations against the same index without re-embedding each time.

Retriever and Search Configuration

- **Retriever:** Your retriever need not be limited to vector similarity search. RapidFire AI accepts any LangChain `BaseRetriever` implementation, which means you can use BM25 retrievers, hybrid retrievers, ensemble retrievers, or any custom retriever you build. This is a meaningful design choice worth experimenting with.
- **Search Configuration:** When using the built-in vector store retriever, you can configure the search algorithm (similarity, similarity with score threshold, or MMR for diversity), top-k, and other parameters. These are all valid knobs for multi-config experimentation.
- **Reranking:** Optional but recommended. RapidFire AI supports cross-encoder rerankers (e.g., `CrossEncoderReranker` from LangChain) that reorder retrieved chunks by relevance before context assembly.

4 Release Packet Content

Apart from this statement, the release packet has the following files:

1. **Documentation Corpus:** A zipped snapshot of the RapidFire AI OSS documentation as plaintext RST files. These files are already clean, structured text with section headers—no binary parsing or format conversion is needed. This is the knowledge base your system must answer questions about.
2. **Released Validation Set:** A set of golden-labeled Q&A pairs in JSON format. Each example has a `question_id` (integer), a `question` (natural language string), a `reference_answer` (string), and a `source_evidence` field containing a list of one or more source evidence locations, each specified as a filename and line number span, e.g., `{"file": "experiment.rst", "lines": [50, 65]}`.
3. **Retrieval Evaluation Script:** A script that computes retrieval metrics (Precision@k, Recall@k, F1@k for specified k) by measuring overlap between your system's retrieved chunks and the ground-truth source evidence locations. A retrieved chunk is considered a hit if it overlaps with at least one ground-truth span for that question.
4. **LLM Judge Prompt:** A prompt template for using a TritonAI-accessible model as an LLM judge to assess generation quality. The judge evaluates 3 metrics:
   - **Correctness** (binary for Pass/Fail): Are the claims in the answer consistent with the reference answer, with no factual errors on the main claim?
   - **Faithfulness** (binary for Pass/Fail): Is every substantive claim in the answer grounded in the retrieved context, or does it introduce unsupported information?
   - **Completeness** (numeric score 0–5): Does the answer cover all relevant aspects the reference answer addresses, or does it miss important information?
5. **About Hidden Test Set:** There will be a hidden test set used for your actual scores based on the same retrieval metrics and the same three generation metrics above, plus two additional hidden LLM judge metrics. Both hidden metrics are binary for Pass/Fail. This prevents overfitting to the released rubric while giving you enough information to design your pipeline thoughtfully.
6. **Leaderboard Formula:** The leaderboard score is a weighted average of retrieval and generation metrics, with each step weighted equally. Binary metrics contribute 1 for Pass and 0 for Fail; graded metrics are normalized to $[0, 1]$ by dividing by 5. The final leaderboard score is in $[0, 1]$, with higher being better.

$$\text{Leaderboard Score} = 0.5 \times \text{Retrieval Score} + 0.5 \times \text{Generation Score}$$

$$\text{Retrieval Score} = \frac{\text{F}_1\text{@5} + \text{Precision@5} + \text{Recall@5}}{3}$$

$$\text{Generation Score} = \frac{1}{5}\left(\text{Correctness} + \text{Faithfulness} + \frac{\text{Completeness}}{5} + \text{Hidden}_1 + \text{Hidden}_2\right)$$

where Correctness, Faithfulness, Hidden_1, and Hidden_2 are binary ($\{0, 1\}$), and Completeness is an integer in $\{0, 1, \ldots, 5\}$.

5 What You Must Produce

5.1 Golden Q&A Evaluation Data

Create at least **20** golden hand-labeled Q&A pairs over the documentation corpus. These pairs are graded on quality and diversity as part of your report/submission grade. They are NOT used as part of the hidden test set or for evaluating other teams. You are free to use LLMs to aid this part but make sure to proofread the final examples.

Each pair must include:

- `question_id`: an integer
- `question`: a natural language question a real user might ask about the documentation
- `reference_answer`: a solid reference answer as a string
- `source_evidence`: a list of one or more source evidence locations, each as `{"file": <filename>, "lines": [<start>, <end>]}`

Follow the same JSON schema as the released validation set.

Your Q&A pairs should cover a variety of question types:

- **Factual lookup** (e.g., "What arguments does `RFGridSearch` accept?")
- **Conceptual explanation** (e.g., "How does the adaptive execution engine differ from sequential execution?")
- **Procedural** (e.g., "How do I set up RapidFire AI for RAG evaluation?")
- **Comparative** (e.g., "What is the difference between `RFGridSearch` and `RFRandomSearch`?")

Quality matters. Trivial questions, questions answerable without the corpus, or questions with ambiguous answers will be flagged during grading.

You may also optionally create additional custom eval metrics beyond those provided. If you do, describe them in your report and explain what failure modes they are designed to catch. While this part is optional, your creativity can help anticipate the hidden LLM judge metrics.

5.2 Automated RAG Pipeline

Your submission must have a `main.py` script that automates your full pipeline. It must be runnable end to end by the TAs as a CLI command taking as input a single input file and outputting a single output file as follows.

```
python3 main.py --input input_filename --output output_filename
```

Internally your script can use `rapidfireai` and/or other Python packages. If any additional package installs are needed to run your code, please explain them in your Project Report and git repository README. The input and output file schemas are as follows:

> **Input File:** A test set JSON file containing questions as a list of dictionary per example, each with two key-value pairs:
>
> - `question_id` key with an integer value
> - `question` key with a string value.

Output File: A JSON file containing examples as a list of dictionary per example, each with the following key-value pairs:

- `question_id` key with an integer value
- `answer` key with a string value.
- `retrieved_context` key with a string value: the full concatenated context text that was passed to the generator LLM, i.e., the serialized retrieved chunks, including any formatting or separators your pipeline applied. This is used by the LLM judge to score Faithfulness. If this field is missing or empty Faithfulness will be 0.
- `sources` key with a list of one or more source evidence locations as value, each specified as `{"file": <filename>, "lines": [<startline>, <endline>]}`. A source file can appear in multiple entries.

Your pipeline must respect the per-query context budget of 2,000 tokens. Do NOT hardcode any answers, paths to intermediate results, or validation set-specific logic. The TAs will run your script on the hidden test set.

5.3 Project Report

Include a succinct project report (say, 4 pages) in PDF format with the following 5 sections:

a. **Data Creation Methodology:** How you designed your Q&A pairs. What question types you targeted, how you ensured quality, and any augmentation or iteration you did on your evaluation data. Any additional LLM judge metrics you tried.
b. **Final Pipeline Architecture:** Your final RAG pipeline design—chunking strategy, embedding model, retrieval method, reranking (if any), prompt design, and how you assembled the context within the budget.
c. **Experimentation Methodology:** What config combinations did you compare using RapidFire AI, why you chose those config knob values, and how you used the multi-config experimentation API, e.g., `RFGridSearch` over chunking sizes, embedding models, and rerankers. You must demonstrate **at least 8** meaningfully distinct configs. Include a table of all config knob details from your RapidFire AI experiments.
d. **Results and Analysis:** Tabulate and/or plot the results of all configs that you tried and explain how you arrived at your final pipeline. What configs worked well and what did not? Why or why not? Which knobs had the biggest impact on retrieval metrics vs. generation metrics? Was there anything that surprised you in the results? What would you do differently or what more will you try if you had a lot more time or budget?
e. **AI Tools Statement:** Brief description of if/how you used AI assistants such as Cursor, Claude Code, OpenClaw, etc. for your work.

5.4 Git Repository

You must set up a private git repo within 24 hours of the project release and add all TAs and the instructor as collaborators. Their GitHub IDs are provided on Piazza.

Your git repo must contain all your code, configuration files, your golden Q&A pairs file, the output file based on our released validation set, the `main.py` file as described above, a `README.md` file, your project report PDF, and a subfolder named `logs` with all your RapidFire AI experiment logs and

metrics.

Reg. the logs, RapidFire AI outputs them as `rapidfire.log`; all metrics files are written to your `experiments_path` directory. Do NOT forget to include these files in your regular commits/pushes. This will help the TAs verify your project progress and also assist with any debugging needed for partial credits during grading.

Important: Your git commit/push history must reflect genuine iterative development over the project duration. Do NOT dump all your code created by an AI tool. Forging your git commit/push history is a violation of academic integrity.

Make sure to read all policies on teams, academic integrity, and git requirements again on the course Projects overview page.

6 Grading Your Submissions

6.1 Auto-Grading Your Code: 6%

Your `main.py` will be auto-graded by our scripts on both the released validation set and on our hidden test set. The output file you provide will be cross-checked to ensure your code produces it correctly on the released validation set. But your project score and all metrics prescribed in the leaderboard formula will be calculated based only on our hidden test set.

If you code fails to produce expected and consistent outputs or if it fails to work properly on our hidden test set, you will lose points on this part accordingly. Likewise, if the scores are much worse than expected based on our pipelines, you will lose points accordingly again.

6.2 Golden Data File: 2%

The TAs will grade your golden Q&A pairs as explained in Section 5.1. This includes running them through our pipelines and cross-checking the generated answers against your reference answers. If your examples are trivial or too repetitive, you will lose points accordingly.

6.3 Project Report Grading: 2%

The TAs will grade your project report using both LLMs and manual reading. You will be evaluated on the depth and rigor of experimental methodology, evidence of systematic experimentation (not ad hoc trial and error), and clarity of reasoning over the results.

6.4 F2F Interview: 10%

The TAs will conduct 10min F2F interviews based on your entire submission. The interview will focus on diving into your approach, results, and counterfactual questions, e.g. "What would happen if you changed X?" or "Why didn't you try Y?" You will NOT have access to electronics/Web/LLMs during the interview but the TAs will.

The outcome of interview will be Pass (10%) or Re-interview. Re-interview will be conducted by the professor, with one of three final outcomes: Pass (10%), Half-pass (5%), or Fail (0). The professor's decision on this is final.

6.5 Extra Credit based on Leaderboard

The leaderboard is NOT part of the base grade. Its purpose is to motivate you to excel in this project above and beyond the expectations. The top **10th percentile** of the teams will receive **2% extra credit** toward the course grand total. The rest of the top **20th percentile** will receive **1% extra credit**.

An intermediate one-time preview of the leaderboard will be made for your benefit roughly a week before the actual due date based on early submissions made till then. No additional feedback will be provided apart from your leaderboard position.

Early submission is strongly encouraged to help you gauge your standing and leave time for iterative improvement. You can resubmit any number of times after that. Your final leaderboard position (for extra credit determination) will be based on your final submission on our hidden test set.

7 Submission Logistics

All submissions must be made via Canvas. Only one submission is needed per team. If you resubmit multiple times, only your final submission will be used. Your submission must be a single zipped folder named `TeamID.zip` (where TeamID is your assigned team ID) with the following files/subfolders:

1. Golden Q&A pairs file as a plaintext JSON file.
2. Output JSON file based on your final pipeline code and the released validation set.
3. Zipped folder of your final git repo (just download it as such). This must include a `main.py`, a `README.md` file, and the subfolder named `logs` at the top level.
4. Project report PDF.

In addition to the above zipped folder via Canvas, make sure to also add all TAs and the instructor as collaborators on your private git repo within 24 hours of the project release.

Important: Experimentation vs. Final Pipeline. Your experimentation process using RapidFire AI's `run_evals`, Interactive Control Operations, etc. can be interactive and documented via your logs and project report. But your final submission code's pipeline must be a *deterministic* automated script that uses your chosen configuration knobs to produce outputs on a given input file. The TAs will grade your outputs by running this final pipeline code—they will NOT be required to replay your entire RapidFire AI experimentation sequence or Interactive Control Operations. In other words, your experimentation methodology and process will be assessed through the logs/metrics (and git history), project report, and F2F interview.

8 Timeline

Milestone	Date
Release Date	Wed, Apr 15
Discussion by TAs	Wed, Apr 15
Git Repo Deadline	Thu, Apr 16, 11:59pm PT
Early Submission for one-time leaderboard preview	Thu, Apr 23, 11:59pm PT
Due Date	Wed, Apr 29, 11:59pm PT
F2F Interviews	Tue, Apr 28 to Wed, May 6

Note: The early submission date allows you to get a one-time private reveal of your leaderboard position. It is based only on your code and our hidden test set. It will only indicate your position among those who have submitted. No additional feedback will be provided on your submission. You can optionally resubmit an improved version by the actual due date. Your final leaderboard position and scores are based on your last submission on our hidden test set.

9 Tips

- **Start with the tutorial notebooks.** Get a working understanding of `rapidfireai` end to end using the tutorial notebooks before you start optimizing for your work. A working baseline for this project use case should be doable within a day. Add complexity and sophistication incrementally. Remember, as Don Knuth said, "Premature optimization is the root of all evil!"
- **Budget your API calls.** Downsample the data as needed to allow you to iterate faster until you have a fully working multi-config experimentation setup. Then move to using the full validation set and your own golden pairs.
- **Understand the context budget constraint.** Yes, this corpus is small enough to fit in many LLMs' context window. But the per-query context budget is an intentional pedagogical constraint to expose you to the retrieval and context engineering skills necessary in practice where document corpora can be massive. Embrace the constraint prudently and be mindful of it in your submission. If you exceed this constraint, your code will be treated as failing and you will lose points accordingly.
- **Remember the retrieved_context field.** Your main.py must emit retrieved_context in each output entry; this is the text the LLM judge uses to score Faithfulness. Students often build a working RAG pipeline but forget to serialize the context into the output JSON. Without this field, Faithfulness will score 0!
- **Use RapidFire AI's multi-config framework intentionally.** Learn to define config groups with `RFGridSearch` or `RFRandomSearch` and compare systematically. Use `get_results` and `get_runs_info` in your scripts as needed between multiple experiments. Use Interactive Control Ops (Stop, Resume, Clone-Modify, etc.) to redirect effort from unpromising configurations to promising ones.
- **Your Q&A pairs and project report are also graded deliverables, NOT an afterthought.** Good evaluation data is hard to create in practice. Invest time in this–it will also help you debug your pipelines. Likewise, make sure to produce a well-organized and neat project report. That will help the TAs understand your work properly and also ease your F2F interview.

- **Prepare for the F2F interview.** Understand every design decision in your pipeline. If you used AI tools to generate code, make sure you can explain that code and its counterfactuals on the spot.
- **Collective responsibility of a team.** If you are in a team of 2, both of you are collectively responsible for what you submit. This also means you must fully know the other's part of the work and its results. By default both members of a team will get same score. But if during the F2F interview the TAs suspect an imbalance among the team members' knowledge, you will be escalated to the professor for a Re-interview and possibly, separate scores.